Votee Wordle Solver

Project Walkthrough — Forward Deployed Engineer [VT26-FDE]

Mayank Chugh | April 2026 | github.com/mayankchugh-learning/Wordle

mayankchugh.jobathk@gmail.com | +852-64851129 | linkedin.com/in/mchugh77

What This Program Does

Task

Automatically solve Wordle puzzles by connecting to the Votee API and making intelligent guesses

`--mode random`

Solve a random word (seed controls which word)

`--mode daily`

Solve today's live daily puzzle

`--mode word`

Solve a specific known word — for testing

The solver picks the best guess → calls API → reads green/yellow/grey → filters candidates → repeats

File Architecture — One File, Clear Sections

CONFIG

BASE_URL, WORD_LIST_URLS, WORD_SIZE, MAX_GUESSES

LOAD WORDS

load_words() — download, merge, fallback

API CALLS

guess_random(), guess_daily(), guess_word()

FILTER LOGIC

evaluate_guess(), filter_words()

GUESS STRATEGY

get_cluster_signature(), pick_best_guess()

SOLVER + CLI

solve(), parse_args(), run_multiple_games()

Main Solving Loop — solve()

1 **`load_words()`**
Download ~20,000 five-letter words from two sources

2 **`pick_best_guess()`**
Choose best word from current candidates

3 **`call API`**
GET /random or /daily or /word/{target}

4 **`read feedback`**
Green / Yellow / Grey per letter position

5 **`all green?`**
If yes → SOLVED. If no → filter candidates

6 **`filter_words()`**
Remove impossible words → repeat from step 2

filter_words() — The 3 Rules

Green — correct

Letter MUST be at this exact position

```
Keep if word[slot] == letter
```

Yellow — present

Letter IS in word but NOT at this slot

```
Keep if letter in word AND word[slot] != letter
```

Grey — absent

Letter NOT in word at all

```
Remove if letter in word (unless also green/yellow)
```

pick_best_guess() — Stage-Aware Strategy

Attempt 1	**Always: CRANE** 5 high-frequency unique letters — maximum first-guess information
Large pool (200+)	**Letter frequency scoring** Score words by sum of letter frequencies across all candidates
Mid pool (10–200)	**Partition scoring** Pick guess that splits candidates into smallest worst-case bucket
Small pool (1–10)	**Endgame + sacrifice** Max shared letters; sacrifice probe for tight clusters like FAINT/SAINT

load_words() — Word List Strategy

Layered fallback design — never fails silently

1	**System dictionary** /usr/share/dict/words — Mac/Linux only	Try first
2	**tabatkins/wordle-list** Official Wordle words — 2,309 real answers	Download
3	**dwyl/english-words** Extended English dictionary — 15,921 words	Merge
4	**Merged result** Deduplicated, sorted — ~20,000 five-letter candidates	Use this
5	**Hardcoded fallback** 35 common words — last resort if all downloads fail	Safety net

API Integration — 3 Endpoints

Base URL: https://wordle.votee.dev:8000

GET /random

guess (required), seed (optional), size (default 5)

Reproducible random word — same seed = same word every time

GET /daily

guess (required), size (default 5)

Today's live daily puzzle — changes every day

GET /word/{word}

word (path, required), guess (required)

Specific known target — used for deterministic testing

```
Response: [{"slot": 0, "guess": "c", "result": "absent"}, {"slot": 1, "guess": "r", "result": "correct"}, ...]
```

Test Results

Command	Word	Guesses / Result	Status
`seed=42`	**WROTE**	5	**SOLVED**
`seed=1`	**FIERY**	4	**SOLVED**
`seed=7`	**ARSON**	4	**SOLVED**
`seed=99`	**CODEX**	3	**SOLVED**
`--games 3`	**STORM, FIERY, SMELT**	3/3 — 100%	**SOLVED**
`--mode word --target crane`	**CRANE**	1	**SOLVED**

Average: 3.5 guesses per game across all seeds tested

Design Decisions

Why CRANE?

5 high-frequency unique letters — C,R,A,N,E. No repeats. Maximum first-guess information.

Why frequency not minimax?

Speed (instant vs expensive), results (4-5 avg vs 3.4 minimax), explainability for clients.

Why merge word lists?

API uses words outside any single public list. Merging prevents zero-candidate failures.

Why sacrifice guess?

FAINT/SAINT/PAINT/TAINT clusters waste 4 guesses without it. One probe collapses the cluster.

Why one file?

Focused assessment — easier to review and run. Production would split into modules.

CLI Arguments + Multi-Game Statistics

Command-line arguments via argparse:

```
python wordle_solver.py                          # default: random seed=42
python wordle_solver.py --mode daily             # today's puzzle
python wordle_solver.py --mode random --seed 99  # specific seed
python wordle_solver.py --mode word --target crane  # known word
python wordle_solver.py --games 5               # run 5 games + stats
```

run_multiple_games() — win rate statistics:

```
def run_multiple_games(n, mode='random'):
    shared_words = load_words()  # load ONCE — not per game
    wins = 0
    for i in range(n):
        result = solve(mode=mode, seed=i, preloaded_words=shared_words)
        if result: wins += 1
    print(f"Final: {wins}/{n} ({100*wins//n}% win rate)")
```

What I Would Improve Next

Unit tests

Test filter_words and evaluate_guess with known repeated-letter cases. Catch regressions automatically.

Pre-computed openers

Offline minimax to find optimal first 2 guesses. Reduce average from 4-5 to 3-4.

Module split

api.py, strategy.py, solver.py, cli.py — better maintainability and testability.

Pattern caching

Cache evaluate_guess results for repeated (guess, target) pairs — faster partition scoring.

Async API calls

Replace requests with httpx async — useful when serving as an API endpoint at scale.

Thank You

Ready to walk through any part of the code in detail

Mayank Chugh | mayankchugh.jobathk@gmail.com | +852-64851129

github.com/mayankchugh-learning/Wordle | linkedin.com/in/mchugh77